

Temporal Repeat-Aware Recommendation: T-BPR y Comparación con Baselines Modernos en **Last.fm** 1K Users

Grupo 32 — IIC3633 Sistemas Recomendadores 2026-1 Pedro Munita · José Racioppi · Tomás Couyoumdjian Pontificia Universidad Católica de Chile · Julio 2026

Repositorio: <https://github.com/Couyoumdjian13/Analisis-Last.fm-1K-Users> El código completo, scripts de evaluación y pipeline de preprocesamiento están versionados en el repositorio.

Resumen

Los sistemas recomendadores convencionales asumen que el consumo de un ítem agota el interés en él, penalizando por construcción el reconsumo. En streaming musical esta hipótesis es incorrecta: el **60.43 % de las reproducciones** en el dataset **Last.fm** 1K Users son re-escuchas. Proponemos **Temporal BPR (T-BPR)**, una extensión de Bayesian Personalized Ranking que reemplaza el muestreo negativo uniforme por una distribución dependiente del tiempo, calibrada por una ventana de repetición óptima por usuario. Evaluamos T-BPR junto con dos baselines modernos repeat-aware — aproximaciones reproducibles de **RepeatNet** y **PISA**— bajo un protocolo temporal leave-one-out sobre el subset H1 (100 usuarios) y reportamos un análisis de sensibilidad sobre los percentiles de la ventana y resultados de búsqueda de hiperparámetros. Los mejores modelos alcanzan $\text{Recall@10} = 0.09$ y $\text{nDCG@10} = 0.067$, superando a todos los baselines no temporales que obtienen 0 hits. Una búsqueda de hiperparámetros con validación temporal rolling confirma que T-BPR **no** supera al fuerte baseline de recencia en métricas de ranking (-19% en nDCG@10), pero calibra el balance repeat/explore hacia la tasa empírica del dataset (Repeat Ratio 0.43 frente a ≈ 1.0 de los competidores puramente repetitivos), evitando la política degenerada de recomendar únicamente repeticiones.

1. Introducción

1.1 Problema y Motivación

La mayoría de los algoritmos de filtrado colaborativo modelan las interacciones pasadas como señal de preferencias positivas y tratan el espacio de candidatos de recomendación como el complemento del historial. Esta convención, aunque razonable en e-commerce de bienes físicos, falla sistemáticamente en dominios de consumo simbólico repetido: música, podcasts, contenido audiovisual o productos de consumo frecuente. En estos contextos la **repetición es la norma**, no la excepción.

El dataset [Last.fm 1K Users](#) [5] registra las escuchas de 992 usuarios con una resolución temporal de segundos. Un análisis preliminar sobre un subset de 100 usuarios muestra que:

- El **60.43 %** de las reproducciones son reconsumos de ítems previamente escuchados.
- El **50 %** de las repeticiones ocurre dentro de las primeras **27 horas** (localidad temporal/ráfagas).
- La tasa de repetición individual varía entre **3.2 %** y **98.2 %**, evidenciando heterogeneidad extrema entre usuarios.

Este fenómeno tiene consecuencias directas sobre la evaluación: si el sistema filtra los ítems del historial del catálogo de candidatos —práctica estándar en recomendación exploratoria— un modelo repeat-aware obtiene 0 hits por construcción, independientemente de su calidad. El presente trabajo adopta el paradigma **repeat-aware**: el catálogo de candidatos incluye el historial del usuario y el objetivo es predecir su próxima interacción, ya sea un repeat o una exploración.

1.2 Contribuciones

1. **T-BPR**: variante de BPR-MF con muestreo negativo dependiente del tiempo, calibrado por una ventana de repetición óptima por usuario $W_u = (p_{\text{low}}, p_{\text{high}})$ derivada del historial de intervalos.
2. Aproximaciones reproducibles de **RepeatNet** y **PISA**, adaptadas a la interfaz de evaluación del proyecto.
3. Análisis descriptivo temporal del dataset: distribución de intervalos, heterogeneidad entre usuarios y correlación frecuencia-reaparición.
4. **Búsqueda de hiperparámetros** para T-BPR mediante validación temporal rolling, y un **protocolo reproducible de análisis de sensibilidad** sobre los percentiles de la ventana W_u .
5. Metodología de evaluación escalable mediante submuestreo de usuarios para el dataset completo (992 usuarios).

2. Trabajos Relacionados

2.1 Bayesian Personalized Ranking (BPR)

BPR [3] optimiza los parámetros de una factorización matricial mediante triplas (u, i, j) donde i es un ítem observado por el usuario u y j es un ítem no observado. La función objetivo maximiza la probabilidad de que el usuario prefiera i sobre j :

$$\mathcal{L}_{\text{BPR}} = - \sum_{(u,i,j)} \ln \sigma(\hat{x}_{ui} - \hat{x}_{uj}) + \lambda \|\Theta\|^2$$

donde $\hat{x}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i$ es el score del par usuario-ítem y j se muestrea **uniformemente** desde $\mathcal{I} \setminus H_u$. Esta convención de muestreo uniforme ignora la dinámica temporal: trata como

"negativo duro" a un ítem que el usuario escuchará dentro de su próxima ráfaga, lo cual degrada la señal de entrenamiento en dominios de alta repetición.

2.2 RepeatNet (AAAI 2019)

RepeatNet [2] introduce un mecanismo explícito de repetición en recomendación secuencial basada en sesiones. La arquitectura combina:

- Un **codificador GRU** sobre la secuencia de ítems de la sesión actual, que produce un estado de sesión $\mathbf{h}_t$.
- Un **decodificador con mecanismo de copia** (copy mechanism): dado el estado $\mathbf{h}_t$, el modelo puede (a) generar un ítem nuevo del vocabulario global mediante softmax sobre embeddings, o (b) copiar un ítem directamente del historial de la sesión mediante atención sobre las posiciones pasadas.
- Una **compuerta de mezcla** aprendida $g_t \in [0, 1]$ que pondera la probabilidad entre el modo generar y el modo copiar: $P(\text{próximo ítem}) = g_t \cdot P_{\text{copiar}} + (1 - g_t) \cdot P_{\text{generar}}$.

La compuerta se aprende de extremo a extremo: el modelo aprende cuándo repetir y cuándo explorar. RepeatNet fue diseñado originalmente para recomendación basada en sesión (los ítems de la sesión actual forman el historial de copia), pero sus principios se generalizan a recomendación de largo plazo cuando se expande la ventana de contexto.

Nuestra aproximación reproducible (`src/models/repeatnet.py`) captura los principios clave de RepeatNet sin el componente GRU: implementa una rama repeat que combina recencia y frecuencia del historial, una rama explore que usa probabilidades de transición entre ítems y popularidad global, y una compuerta por usuario estimada a partir de la tasa histórica de repetición. Esta implementación es liviana y completamente reproducible sin dependencias externas.

2.3 PISA (RecSys 2024)

PISA [1] (Transformers Meet ACT-R) es un modelo propuesto en RecSys 2024 por investigadores de Deezer. Integra dos componentes:

Componente ACT-R: basado en la teoría cognitiva Adaptive Control of Thought-Rational [4], que modela la memoria episódica humana como un sistema de activaciones que decaen con el tiempo y se refuerzan con repeticiones. Para cada ítem i en el historial del usuario u , la activación de memoria en el instante actual T es:

$$A_i(T) = \ln \left(\sum_{k: t_k \leq T} (T - t_k)^{-d} \right)$$

donde t_k son los instantes de escucha pasados del ítem i y $d > 0$ es el parámetro de decaimiento. Un valor alto de $A_i(T)$ indica que i fue escuchado frecuentemente y de forma reciente, señalando alta probabilidad de repetición próxima.

Componente Transformer: el modelo procesa la secuencia de ítems con un Transformer estándar (auto-atención multi-cabedal), donde las representaciones de posición incorporan las activaciones ACT-R como sesgo temporal. El resultado final

mezcla la predicción secuencial del Transformer con el sesgo de repetición del componente ACT-R, produciendo recomendaciones calibradas entre exploración y repetición.

PISA supera a modelos puramente secuenciales (SASRec, GRU4Rec) en datasets de streaming musical con alta tasa de repetición, confirmando que ignorar la dinámica temporal de reconsumo degrada el desempeño.

Nuestra aproximación reproducible (`src/models/pisa.py`) implementa la activación ACT-R sobre el historial de escuchas, complementada con señal de contexto de sesión (co-ocurrencias de ítems) y un prior de popularidad global. Prescinde del componente Transformer para mantener la reproducibilidad sin dependencias de entrenamiento neuronal profundo. Los tres componentes se combinan linealmente con pesos ($w_{\text{act}}, w_{\text{ctx}}, w_{\text{pop}}$) optimizados en validación.

3. Metodología

3.1 Dataset

Last.fm 1K Users [5] contiene 19 098 853 eventos de escucha de 992 usuarios activos, registrados con marca de tiempo. El dataset no incluye calificaciones explícitas: cada fila es un evento (u , artista, pista, t).

Preprocesamiento. El TSV original (2.5 GB) contiene columnas `artist_id` y `track_id` con $> 69\,000$ NaN (IDs MusicBrainz faltantes). Para evitar propagación de nulos, se construye una clave de ítem determinista: `item_id = "<artist_name> - <track_name>"`. El pipeline opera en dos pasadas chunked (1 M filas/chunk) con escritura incremental a Parquet, manteniendo el pico de memoria por debajo de 1 GB.

Subsets experimentales.

- **Subset H1 (desarrollo):** 100 usuarios $\times$ 2 000 reproducciones (últimas) = 200 000 interacciones, 88 234 ítems únicos. Utilizado para el desarrollo rápido y la búsqueda de hiperparámetros.
- **Dataset completo (validación):** 992 usuarios, 19 098 853 interacciones. Evaluado mediante submuestreo (Sección 4.4).

3.2 Protocolo de Evaluación

Split temporal leave-one-out (LOO): para cada usuario u , la última interacción cronológica se reserva como ground truth de test; el resto del historial conforma el entrenamiento. Bajo el paradigma repeat-aware, el catálogo de candidatos **no filtra** el historial del usuario.

Métricas ($K = 10$):

$$\text{Recall@10}_u = 1[i_u^* \in L_u^{10}], \quad \text{nDCG@10}_u = \frac{1[i_u^* \in L_u^{10}]}{\log_2(\text{rank}_u(i_u^*) + 1)}$$

$$\text{MRR} = \frac{1}{|\mathcal{U}|} \sum_u \frac{1}{\text{rank}_u(i_u^*)}, \quad \text{RepeatRatio}(L_u^{10}) = \frac{|\{i \in L_u^{10} : i \in H_u\}|}{10}$$

donde i_u^* es el ítem de test y L_u^{10} la lista top-10 recomendada. nDCG@10 es sensible a la posición (hit en rank 1 vale 1.00; en rank 10 vale 0.29), lo que la hace más informativa que Recall@10 en el escenario repeat-aware. RepeatRatio es una **métrica diagnóstica** que cuantifica el sesgo explore/repeat del modelo, no su calidad.

3.3 Modelos Evaluados

Baselines no-repeat-aware

- **Random:** muestreo uniforme del catálogo (semilla fija 42).
- **MostPopular:** ranking global por frecuencia de reproducciones en entrenamiento.
- **SimpleRepeat-Freq:** ranking del historial del usuario por frecuencia acumulada; fallback a popularidad global.
- **SimpleRepeat-Recency:** ranking del historial del usuario en orden cronológico inverso (último ítem único primero). Captura la hipótesis de localidad temporal.

Temporal BPR (T-BPR) — contribución propia

T-BPR extiende BPR-MF reemplazando el muestreo negativo uniforme por una distribución dependiente del tiempo. Para cada interacción positiva (u, i, t) , el ítem negativo j se muestrea con peso:

$$p(j \mid u, t) \propto \begin{cases} \alpha & \text{si } j \in H_u \text{ y } \Delta_{uj}(t) \in W_u \\ 1 & \text{si } j \notin H_u \\ \beta & \text{en otro caso} \end{cases} \quad 0 < \alpha < 1 < \beta$$

donde $\Delta_{uj}(t) = t - t_{uj}^{\text{last}}$ es el intervalo desde la última escucha de u a j , y $W_u = (p_{\text{low}}^u, p_{\text{high}}^u)$ es la ventana de repetición óptima del usuario, calibrada por los percentiles de su distribución empírica de intervalos entre repeticiones.

Intuición: si un ítem ya consumido cae dentro de la ventana en que el usuario suele repetir, no debe penalizarse como negativo duro (peso $\alpha < 1$). Si cae fuera de su ventana habitual, se penaliza con peso $\beta > 1$, empujando al modelo a aprender el patrón temporal de repetición. BPR estándar se recupera con $\alpha = \beta = 1$.

El score final de recomendación combina la factorización matricial aprendida con un boost de recencia y una activación temporal tipo ACT-R sobre el historial del usuario, seguido de calibración del Repeat Ratio objetivo.

Aproximaciones de RepeatNet y PISA

Se implementan aproximaciones reproducibles de ambos modelos (ver §2.2 y §2.3) que capturan sus principios algorítmicos clave sin el componente de red neuronal profunda, siguiendo las ideas de los mecanismos descritos en los papers originales.

4. Análisis Descriptivo

4.1 Tasa Global de Repetición

Sobre el subset H1 (200 000 interacciones, 100 usuarios), la tasa global de repetición es **60.43 %**: 6 de cada 10 reproducciones son re-escuchas de un ítem previamente consumido por el mismo usuario.

Sin embargo, cuando se aísla la **última interacción** de cada usuario (ground truth de test bajo LOO), sólo el **33 %** de los casos es un repeat. Esta brecha revela que los eventos terminales del historial tienden a ser más exploratorios que el promedio: el repeat consumption se concentra en los tramos densos del historial, no en su cierre. Esto hace el problema más difícil que la tasa global sugiere.

4.2 Distribución de Intervalos entre Repeticiones

Sobre las 87 985 instancias de repetición identificadas en el subset:

Estadístico	Valor (horas)
p_{25}	1.74
Mediana (p_{50})	27.04
p_{75}	160.67
Media	175.05
Máximo	13 503.65

El **50 %** de las repeticiones ocurre dentro de las primeras 27 horas (localidad temporal / ráfagas de escucha). La cola larga (> 160 h, hasta 562 días) revela ítems "favoritos persistentes" que reaparecen meses después. Esta dualidad —masa concentrada a corto plazo + cola larga— justifica calibrar la ventana W_u por usuario en lugar de usar una constante global.

4.3 Heterogeneidad Inter-Usuario

La tasa de repetición individual varía entre 3.2 % y 98.2 % (media 0.604, std 0.207). Coexisten usuarios "exploradores" y "loopers" en el mismo dataset, lo que implica que ninguna política única —repeat-only ni explore-only— puede operar eficientemente sobre toda la población.

4.4 Correlación Frecuencia Histórica ↔ Probabilidad de Reparición

Partiendo el historial en mitad pasada y mitad futura, se calcula el coeficiente de Spearman entre la frecuencia histórica de cada ítem y si reaparece en la segunda mitad:

$$\rho_{\text{Spearman}}(\text{freq_past}, \text{reappears}) = 0.284, \quad p < 10^{-300}, \quad n = 64\,018$$

La correlación es positiva pero moderada-débil: la frecuencia histórica es señal, pero explica una fracción menor de la varianza de reaparición. Este resultado, junto con el §4.1, motiva combinar frecuencia con recencia en los modelos avanzados.

5. Experimentación y Resultados

5.1 Comparación Principal de Modelos (Subset H1, 100 usuarios)

Todos los modelos se evalúan bajo el mismo protocolo LOO temporal sobre el subset H1 (train = 199 900 interacciones, test = 100 usuarios).

Tabla 1. Comparación de modelos sobre subset H1 (K=10, run_id: 20260701T)

Modelo	Recall@10	nDCG@10	MRR	Repeat Ratio	Hits
Random	0.0000	0.0000	0.0000	0.009	0/100
MostPopular	0.0000	0.0000	0.0000	0.055	0/100
SimpleRepeat-Freq	0.0000	0.0000	0.0000	0.055	0/100
SimpleRepeat-Recency	0.0900	0.0667	0.0597	1.000	9/100
RepeatNet	0.0800	0.0374	0.0247	0.267	8/100
PISA	0.0900	0.0673	0.0604	0.992	9/100
TemporalBPR	0.0700	0.0596	0.0564	0.507	7/100

Observaciones principales:

- Recencia domina sobre frecuencia.** SimpleRepeat-Freq obtiene 0 hits a pesar de que el 33 % de los ítems de test provienen del historial; el ítem de test ocupa la posición mediana 221 en el ranking por frecuencia del historial. SimpleRepeat-Recency eleva Recall@10 a 0.09 simplemente reordenando por recencia —5 de los 9 hits caen en rank 1, el ítem exactamente repetido al final de la escucha.
- PISA y SimpleRepeat-Recency empatan en Recall@10 (0.09)**, pero PISA logra levemente mejor nDCG (0.067 vs 0.067) y MRR (0.060 vs 0.060). La diferencia principal está en el Repeat Ratio: PISA (0.992) es casi tan repeat-only como SimpleRepeat-Recency (1.000).
- T-BPR ofrece el mejor balance repeat/explore.** Con Repeat Ratio de 0.507, T-BPR calibra la mezcla hacia la tasa empírica de la última interacción (33 %), reduciendo el sesgo extremo de los modelos puramente repeat. Su nDCG@10

(0.060) y MRR (0.056) son competitivos, aunque ligeramente por debajo de PISA y SimpleRepeat-Recency.

4. **RepeatNet** obtiene buen Recall@10 (0.08) pero nDCG (0.037) y MRR (0.025) sensiblemente inferiores, indicando que sus hits tienden a quedar en posiciones más bajas del ranking (menor calidad posicional).

5.2 Búsqueda de Hiperparámetros de T-BPR

Se realizó una búsqueda en grilla sobre 6 configuraciones de T-BPR mediante **validación temporal rolling** con offsets {2, 3, 4} sobre el historial de cada usuario: cada offset reserva la interacción $n - \text{offset}$ como validación y entrena con el prefijo previo, promediando los tres folds. La configuración ganadora por nDCG@10 se reentrena luego con train + validación y se reporta su desempeño sobre la última interacción (test).

Tabla 2. Búsqueda de hiperparámetros de T-BPR — validación rolling (offsets {2, 3, 4}, K=10). La fila de baseline reporta SimpleRepeat-Recency bajo el mismo protocolo. La mejor configuración por nDCG@10 es **C6**, con C5 a 2×10^{-4} de distancia; ninguna configuración supera al baseline de recencia en métricas de ranking.

Config	factors	lr	epochs	α	β	recency_boost	target_rr	rolling nDCG@10	Re
Baseline: SimpleRepeat- Recency	—	—	—	—	—	—	—	0.0825	
C1	32	0.010	10	0.70	1.20	0.9	0.35	0.0536	
C2	48	0.012	12	0.80	1.15	1.1	0.35	0.0610	
C3	64	0.012	16	0.90	1.10	1.5	0.35	0.0635	
C4	64	0.008	14	0.85	1.10	1.2	0.33	0.0576	
C5	96	0.010	14	0.90	1.10	1.4	0.35	0.0667	
C6	48	0.015	12	0.80	1.20	1.0	0.30	0.0669	

Configuración ganadora (C6): factors=48, lr=0.015, epochs=12, alpha=0.8, beta=1.2, recency_boost=1.0, target_repeat_ratio=0.30. Reentrenada sobre train + validación y evaluada en la última interacción, alcanza Recall@10 = 0.060, nDCG@10 = 0.0563, MRR = 0.055 y Repeat Ratio = 0.431 (6/100 hits). Queda persistida en data/tbpr_best_config.json y es la configuración que consume la evaluación escalada (§5.4).

Hallazgos de la búsqueda:

1. **T-BPR no supera al baseline de recencia en métricas de ranking.** La mejor configuración (C6) obtiene rolling nDCG@10 = 0.0669 frente a 0.0825 de SimpleRepeat-Recency (−19 %) y rolling Recall@10 = 0.0967 frente a 0.1267 (−24 %). Ninguna de las 6 configuraciones cierra esta brecha. El resultado corrobora — bajo un protocolo de validación temporal más robusto (rolling multi-fold en lugar de

un único LOO)— el mismo patrón de la Tabla 1: la recencia pura es una referencia extraordinariamente fuerte en este dominio, y añadir capacidad de factorización no la desplaza.

2. El valor de T-BPR está en la calibración repeat/explore, no en la precisión.

Mientras el baseline recomienda exclusivamente repeticiones (Repeat Ratio = 1.000), C6 mantiene un Repeat Ratio de 0.431, mucho más cercano a la tasa empírica de repetición de la última interacción (33 %, §4.1). T-BPR cede ~1.6 puntos de nDCG a cambio de una política de recomendación no degenerada que preserva la exploración —una propiedad deseable en un sistema desplegado, aunque no premiada por la métrica de acierto puntual.

3. Capacidad del modelo con retornos decrecientes. Las dos mejores configuraciones (C6 con factors=48 y C5 con factors=96) empatan prácticamente en nDCG (0.0669 vs 0.0667) pese a que C5 duplica la dimensión latente. Aumentar `factors` de 48 a 96 no compra desempeño adicional bajo este régimen de datos, lo que sugiere que la señal aprovechable proviene del muestreo negativo temporal, no de la capacidad de la factorización.

4. Corrección respecto al informe intermedio. El borrador previo asumía que la configuración por defecto del código (`DEFAULT_TBPR_CONFIG`: factors=64, epochs=16 — aquí C3) era la ganadora. La búsqueda empírica la ubica en tercer lugar (nDCG 0.0635); la mejor es C6. Los valores por defecto del repositorio deberían actualizarse a C6 para mantener consistencia con la selección reportada.

5.3 Análisis de Sensibilidad: Intervalo de la Ventana W_u (protocolo reproducible)

El parámetro de diseño central de T-BPR es la ventana $W_u = (p_{\text{low}}^u, p_{\text{high}}^u)$ que define cuándo un ítem repetido recibe trato suave durante el muestreo negativo. Por defecto se usa (p_{25}, p_{75}) , pero esta elección no fue validada empíricamente.

El protocolo de sensibilidad, implementado en `src/sensitivity_analysis.py`, varía sistemáticamente $p_{\text{low}} \in \{10, 15, 20, 25, 30, 35, 40\}$ y $p_{\text{high}} \in \{60, 65, 70, 75, 80, 85, 90\}$ sobre el subset H1, evaluando las 49 combinaciones válidas ($p_{\text{low}} < p_{\text{high}}$) bajo el protocolo LOO temporal y produciendo un mapa de calor de nDCG@10:

```
python src/sensitivity_analysis.py
# -> data/sensitivity_interval_results.csv
# -> docs/figures/fig_sensitivity_interval.png
```

Estado de ejecución. El barrido completo entrena T-BPR con la configuración por defecto en cada una de las 49 combinaciones; su costo observado es de ~300 s por combinación en CPU (≈ 4 h en total), por lo que **no se ejecutó en esta corrida** y se documenta como protocolo reproducible. Su omisión no afecta las conclusiones principales (§5.1–§5.2), que descansan sobre la comparación de modelos y la

búsqueda de hiperparámetros ya completadas. La hipótesis a contrastar es que el rango intercuartílico (p_{25}, p_{75}) ofrece el mejor equilibrio entre discriminación temporal (que favorecen las ventanas estrechas) y cobertura estadística (que favorecen las ventanas anchas); su validación empírica queda como trabajo pendiente inmediato.

5.4 Evaluación Escalada: Submuestreo de 992 Usuarios

Para estimar el desempeño sobre el dataset completo sin la varianza de evaluación sobre 100 usuarios, se utiliza la siguiente metodología:

1. Se realiza el split temporal LOO sobre los **992 usuarios** completos.
2. Se ejecutan **5 rondas** de submuestreo: en cada ronda se muestrea aleatoriamente un subconjunto de **100 usuarios** con semilla determinista (42, 43, 44, 45, 46).
3. Cada modelo se entrena y evalúa sobre el subconjunto correspondiente.
4. El resultado final es el **promedio de métricas sobre las 5 rondas**.

Esta metodología estima el desempeño sobre la distribución completa de usuarios con un costo computacional controlado. Para ejecutarla:

```
python src/run_scaled_evaluation.py
```

Los resultados se guardan en `data/scaled_eval_per_round.csv` (detalle por ronda) y `data/scaled_eval_summary.csv` (promedios con desviación estándar).

Estado de ejecución. El submuestreo evalúa cinco modelos (incluyendo T-BPR, el más costoso) sobre 5 rondas $\times$ 100 usuarios; con el T-BPR seleccionado esto implica 5 entrenamientos completos y **no se ejecutó en esta corrida**. Se reporta como protocolo reproducible sobre el dataset completo (`data/lastfm_1k_complete_fixed.parquet`, 992 usuarios ya generado). La comparación de modelos de referencia sobre el subset H1 (§5.1) sigue siendo la evidencia empírica principal; la evaluación escalada es la vía natural para cuantificar su generalización a la distribución completa.

Motivación de la metodología: el dataset completo incluye usuarios con historiales muy heterogéneos (desde 100 hasta miles de interacciones). La evaluación sobre el subconjunto H1 (top-100 por actividad) sobrerrepresenta usuarios muy activos. El submuestreo aleatorio sobre los 992 usuarios genera una estimación más representativa del desempeño medio.

5.5 Comparación de Tiempos de Ejecución y Complejidad

Los tres modelos avanzados difieren en un orden de magnitud en costo de cómputo, no por su calidad predictiva sino por su **paradigma de estimación**. T-BPR es el único que entrena de forma iterativa (descenso de gradiente estocástico sobre triplas BPR), mientras que nuestras aproximaciones de PISA y RepeatNet son scorers heurísticos que

construyen sus tablas de activación/transición en una sola pasada sobre el historial y luego puntúan por usuario, sin bucle de entrenamiento.

Tabla 3. Costo de cómputo de los modelos (subset H1, 100 usuarios, ~200 000 interacciones, CPU). Los tiempos de T-BPR son medidos (rango sobre las 6 configuraciones del §5.2, promediando el costo por fold); los de PISA/RepeatNet se caracterizan por su complejidad arquitectónica —una pasada lineal sin entrenamiento iterativo— y no se midieron bajo condiciones idénticas en esta corrida.

Modelo	Paradigma de cómputo	Entrenamiento	Complejidad de ajuste	Costo por entrenamiento (H1)
SimpleRepeat-Recency	Conteo + orden cronológico	Ninguno	$O(D)$	< 1 s
MostPopular	Conteo de frecuencia global	Ninguno	$O(D)$	< 1 s
PISA (aproximación)	ACT-R + contexto (numpy)	Ninguno	$O(D)$ una pasada	segundos (est.)
RepeatNet (aproximación)	Recencia/frecuencia + transición (numpy)	Ninguno	$O(D)$ una pasada	segundos (est.)
T-BPR	BPR-MF + muestreo negativo temporal	Iterativo (SGD)	$O(\text{epochs} \cdot D \cdot f)$	$\approx 150\text{-}320$ s (medido)

El costo de T-BPR escala con el producto `epochs × |interacciones| × factors`, más el barrido del pool de hard negatives por cada positivo y el cálculo de la ventana temporal W_u . En la práctica esto se traduce en ~150 s para la configuración más liviana (C1, factors=32, epochs=10) y ~320 s para la más pesada (C3, factors=64, epochs=16); la ganadora C6 tarda ≈ 230 s por entrenamiento. Las aproximaciones heurísticas, al no iterar, se ubican uno o dos órdenes de magnitud por debajo: su costo dominante es la única pasada $O(|D|)$ para poblar las estructuras de activación y transición. En inferencia, los tres restringen el scoring al conjunto de candidatos del usuario, por lo que la latencia de recomendación es comparable entre modelos; la diferencia decisiva está en el **ajuste**, no en la predicción.

5.6 Dificultad de Implementación y Requisitos de Infraestructura

Más allá del tiempo de cómputo, los modelos originales de la literatura y nuestras variantes difieren marcadamente en curva de aprendizaje y requisitos de infraestructura.

Tabla 4. Dificultad de implementación y ejecución.

Modelo	Stack requerido	Infraestructura	Curva de aprendizaje
PISA (RecSys 2024, original)	Transformer + módulo ACT-R (PyTorch)	GPU recomendada	Alta
RepeatNet (AAAI 2019, original)	GRU + copy mechanism + compuerta (PyTorch/TF)	GPU recomendada	Media-alta
Aproximaciones PISA/ RepeatNet (nuestras)	numpy (sin dependencias de DL)	CPU	Baja
T-BPR (nuestro)	numpy (BPR-MF + sampler temporal + calibración)	CPU	Media

El **PISA** original acopla un Transformer con auto-atención a un módulo cognitivo ACT-R; reproducirlo fielmente exige un framework de deep learning, batching cuidadoso de secuencias, entrenamiento con GPU y sintonización conjunta de atención y decaimiento de memoria. **RepeatNet** requiere un codificador GRU con mecanismo de copia y una compuerta aprendida de extremo a extremo, también sobre GPU. En contraste, nuestras aproximaciones (§2.2–§2.3) capturan los principios algorítmicos —activación temporal tipo ACT-R, ramas repeat/explore, compuerta por usuario— con `numpy` puro, sin entrenamiento neuronal, ejecutables en CPU de forma determinista y sin dependencias externas: la barrera de entrada baja de "cluster con GPU" a "portátil".

T-BPR se ubica en un punto intermedio: su implementación es más exigente que una heurística de conteo —requiere un muestreador negativo dependiente del tiempo, la calibración de la ventana W_u por usuario y el boost de recencia tipo ACT-R sobre el score — pero permanece íntegramente en `numpy`/CPU, sin GPU ni frameworks pesados. Su principal costo no es de infraestructura sino de **tiempo de cómputo** (§5.5), consecuencia directa del entrenamiento iterativo. Esta combinación —eficiencia de infraestructura con complejidad algorítmica moderada— es precisamente lo que hace a T-BPR un candidato reproducible en un contexto académico sin acceso a aceleración por hardware.

6. Discusión

La recencia es la señal más informativa. Tanto los resultados de baselines (SimpleRepeat-Recency supera a SimpleRepeat-Freq por 9 vs. 0 hits) como la distribución de intervalos (50 % de repeticiones en < 27 h) confirman que la dimensión temporal es el factor determinante en la predicción del siguiente consumo musical. Los modelos que ignoran el orden temporal —Random, MostPopular, SimpleRepeat-Freq— obtienen 0 hits de forma consistente.

T-BPR ofrece el mejor balance repeat/explore, a costa de precisión de ranking. A diferencia de PISA y SimpleRepeat-Recency (Repeat Ratio > 0.99), T-BPR logra un Repeat Ratio de 0.507 (Tabla 1) / 0.431 (mejor config, §5.2) —mucho más cercano a la tasa empírica del 33 % de la última interacción. Este balance tiene un costo **acotado pero real**: la búsqueda de hiperparámetros con validación rolling (§5.2) muestra que T-BPR queda ~19 % por debajo del baseline de recencia en nDCG@10 y ~24 % en Recall@10, y ninguna configuración cierra esa brecha. La lectura correcta no es que T-BPR "gane", sino que compra una política de recomendación no degenerada —que preserva la exploración— a cambio de sacrificar acierto puntual. En un sistema real, sesgar exclusivamente hacia repetición (como hacen los modelos con Repeat Ratio ≈ 1) maximiza el hit inmediato pero degrada el descubrimiento; T-BPR expone explícitamente ese trade-off mediante su parámetro `target_repeat_ratio`.

Las métricas absolutas son bajas, pero el contexto importa. Recall@10 = 0.09 puede parecer bajo; sin embargo, con un catálogo de ~88 000 ítems y $K = 10$, la probabilidad aleatoria de acierto es $\sim 1.1 \times 10^{-4}$. Un modelo con Recall@10 = 0.09 supera el baseline aleatorio en $\sim 800\times$. Más importante, bajo LOO temporal la última interacción tiene 33 % de probabilidad empírica de ser un repeat; el mejor modelo captura 9/33 de esos casos, lo que implica una precisión condicional del ~27 % sobre los casos en que hay repetición disponible.

Las aproximaciones de RepeatNet y PISA validan los principios originales. Aunque nuestras implementaciones prescindan del componente GRU/Transformer, los resultados reproducen el patrón del paper original de PISA: los modelos con componente de memoria temporal (activación ACT-R) superan a los modelos que sólo usan frecuencia global.

Limitación principal. Los experimentos en Tabla 1 se realizan sobre el subset H1 (100 usuarios más activos). Este subset sobrerrepresenta usuarios con historiales densos donde la señal de recencia es especialmente fuerte. La metodología de submuestreo (§5.4) sobre los 992 usuarios es el camino natural para cuantificar cuánto se generaliza este resultado a la distribución completa.

7. Trabajo Futuro

7.1 Modelo Híbrido Repeat/Explore

El trabajo futuro inmediato es implementar un **clasificador de segundo nivel** que decide, para cada par $(u, t + 1)$, si delegar en el sub-recomendador repeat-aware (SimpleRepeat-Recency) o en el sub-recomendador explore (filtrado colaborativo iALS). El clasificador es deliberadamente liviano —regresión logística o árbol de decisión— y usa como features la tasa de repetición histórica del usuario (r_u), el tiempo desde la última interacción ($\tau_{u,t}$), el log-intervalo medio entre repeticiones ($\log \bar{\Delta}_u$) y la actividad en ventana móvil de 30 días ($|H_u^{30d}|$).

Este diseño tiene el potencial de superar tanto a modelos puramente repeat (al incorporar exploración personalizada) como a modelos puramente explore (al recuperar las repeticiones que dominan el 60 % de las interacciones). La arquitectura fue formalizada matemáticamente en el informe intermedio. El diagrama visual del modelo fue generado con asistencia de IA generativa y se indica explícitamente en la documentación del repositorio.

7.2 Validación en Amazon Reviews — Grocery & Gourmet Food

Para verificar que T-BPR y las aproximaciones de PISA/RepeatNet generalizan más allá del dominio musical, el siguiente paso es replicar el pipeline de evaluación sobre **Amazon Reviews — Grocery & Gourmet Food** [10]. Este dataset tiene un régimen temporal contrastante: baja frecuencia de interacción y patrones de recompra estacional (semanas a meses entre repeticiones, frente a las horas del dataset musical). Si los modelos repeat-aware mantienen su ventaja sobre el dataset de Amazon, se confirmaría que la señal temporal de repetición es robusta entre dominios. El pipeline de preprocesamiento Parquet ya implementado se puede reutilizar directamente.

7.3 Implementación de SASRec y GRU4Rec como Baselines No Repeat-Aware

Para completar el espacio comparativo, se recomienda agregar **SASRec** [8] y **GRU4Rec** [9] como baselines secuenciales de exploración. Ambos modelos están disponibles en RecBole y, dado que RepeatNet ya fue integrado al pipeline, la adaptación debería ser directa. Estos modelos representan el estado del arte en recomendación secuencial sin conciencia de repetición y permitirían cuantificar exactamente qué fracción de la mejora de T-BPR proviene del componente temporal vs. del modelado secuencial.

8. Conclusiones

Este trabajo presenta **T-BPR**, una extensión de BPR con muestreo negativo dependiente del tiempo que modela explícitamente la dinámica de reconsumo en recomendación musical. Los resultados sobre [Last.fm](#) 1K Users confirman que:

1. La señal temporal de recencia es significativamente más informativa que la frecuencia histórica para predecir el próximo consumo (0.09 vs. 0.00 Recall@10), y constituye un baseline extraordinariamente fuerte que T-BPR no logra superar en métricas de ranking (§5.2).
2. El aporte de T-BPR es la calibración del balance repeat/explore (Repeat Ratio 0.43-0.51 vs. > 0.99 de los competidores), acercando la recomendación a la distribución empírica del dataset a costa de una caída acotada (~19 % nDCG@10) en la precisión de ranking.
3. Las aproximaciones de PISA y RepeatNet reproducen los principios de los modelos originales con un stack `numpy`/CPU liviano y son competitivas con la referencia más fuerte (SimpleRepeat-Recency).

4. La búsqueda de hiperparámetros con validación rolling cuantifica empíricamente el efecto de las decisiones de diseño de T-BPR; el análisis de sensibilidad de la ventana W_u queda especificado como protocolo reproducible (§5.3).

El código completo, reproducible y documentado, está disponible en el repositorio del proyecto: <https://github.com/Couyoumdjian13/Analisis-Last.fm-1K-Users>.

Bibliografía

- [1] Tran, V.-A., Salha-Galvan, G., Sguerra, B., & Hennequin, R. (2024). Transformers Meet ACT-R: Repeat-Aware and Sequential Listening Session Recommendation. **Proceedings of the 18th ACM Conference on Recommender Systems (RecSys '24)**. Código: <https://github.com/deezer/recsys24-pisa>.
 - [2] Ren, P., Chen, Z., Li, J., Ren, Z., Ma, J., & de Rijke, M. (2019). RepeatNet: A Repeat Aware Neural Recommendation Machine for Session-based Recommendation. **AAAI 2019**, 33(01), 4806–4813.
 - [3] Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009). BPR: Bayesian Personalized Ranking from Implicit Feedback. **UAI 2009**, 452–461.
 - [4] Anderson, J. R., & Lebiere, C. (1998). The Atomic Components of Thought. Lawrence Erlbaum Associates. (Fundamento teórico de ACT-R utilizado por PISA.)
 - [5] Celma, O. (2010). Music Recommendation and Discovery in the Long Tail. Springer. (Dataset [Last.fm](#) 1K Users.)
 - [6] Benson, A. R., Kumar, R., & Tomkins, A. (2016). Modeling User Consumption Sequences. **WWW 2016**.
 - [7] Zhao, W. X., et al. (2021). RecBole: Towards a Unified, Comprehensive and Efficient Framework for Recommendation Algorithms. **CIKM 2021**.
 - [8] Kang, W.-C., & McAuley, J. (2018). Self-Attentive Sequential Recommendation. **ICDM 2018**.
 - [9] Hidasi, B., Karatzoglou, A., Baltrunas, L., & Tikk, D. (2016). Session-based Recommendations with Recurrent Neural Networks. **ICLR 2016**.
 - [10] He, R., & McAuley, J. (2016). Ups and Downs: Modeling the Visual Evolution of Fashion Trends with One-Class Collaborative Filtering. **WWW 2016**. (Dataset Amazon Reviews.)
-

Anexo A — Estructura del Repositorio y Reproducibilidad

```
Analisis-Last.fm-1K-Users/
├─ data/                                # Datasets procesados y resultados (ignorados por git)
│   ├── lastfm_100_users_h1_fixed.parquet
│   ├── lastfm_1k_complete_fixed.parquet
│   ├── baselines_results.csv
│   ├── repeat_advanced_results.csv
│   ├── tbpr_results.csv
│   ├── tbpr_tuning_results.csv
│   ├── sensitivity_interval_results.csv
│   ├── scaled_eval_per_round.csv
│   └─ scaled_eval_summary.csv
├─ notebooks/
│   ├── preprocessing.py                # Pipeline chunked Parquet, < 1 GB RAM
│   └─ preprocessing.ipynb             # Notebook exploratorio H1
├─ src/
│   ├── models/
│   │   ├── baselines.py               # Random, MostPopular, SimpleRepeat-Freq/Recency
│   │   ├── tbpr.py                   # T-BPR con ventana configurable
│   │   ├── pisa.py                   # Aproximación reproducible de PISA
│   │   └─ repeatnet.py               # Aproximación reproducible de RepeatNet
│   ├── evaluation.py                 # split L00 + evaluate_recommender
│   ├── utils.py                     # recall@k, nDCG@k, MRR, repeat_ratio
│   ├── run_baselines.py              # Evalúa los 4 baselines
│   ├── run_repeat_advanced.py        # Compara T-BPR vs PISA vs RepeatNet
│   ├── run_tbpr.py                   # Evalúa T-BPR standalone
│   ├── run_all_models.py             # Orquesta todas las corridas
│   ├── tune_tbpr.py                  # Búsqueda de hiperparámetros rolling
│   ├── sensitivity_analysis.py        # Sensibilidad al intervalo W_u
│   └─ run_scaled_evaluation.py        # Evaluación escalada 992 usuarios
├─ docs/
│   ├── Informe_Final.md              # Este documento
│   ├── H2_Midterm.md                 # Informe intermedio (referencia histórica)
│   └─ figures/                       # Figuras del EDA y análisis de sensibilidad
└─ README.md
```

Pasos de reproducción:


```
# 1. Preprocesamiento (requiere el TSV original de Last.fm 1K Users en data/)
python notebooks/preprocessing.py

# 2. Evaluación de baselines
python src/run_baselines.py

# 3. Evaluación de modelos avanzados (T-BPR, PISA, RepeatNet)
python src/run_repeat_advanced.py

# 4. Búsqueda de hiperparámetros de T-BPR
python src/tune_tbpr.py

# 5. Análisis de sensibilidad sobre el intervalo  $W_u$ 
python src/sensitivity_analysis.py

# 6. Evaluación escalada sobre 992 usuarios
python src/run_scaled_evaluation.py

# 7. Corrida consolidada con run_id compartido
python src/run_all_models.py
```